

TEORÍA (8 puntos)

Apellidos y nombre:

Pregunta 1. Explica en qué consiste una prueba de pares

Pregunta 2. Indica al menos dos beneficios de los entornos automatizados de ejecución de pruebas o «test harness»

Pregunta 3. ¿En qué consiste el «Análisis estático de código»?

Pregunta 4. Describe brevemente la técnica de estimación basada en la Extrapolación

Pregunta 5. Enumera las 7 fases del proceso estructurado de TMAP

Pregunta 6. Distingue las posibles formas de ejecutar las pruebas.

Pregunta 7. En general, ¿por qué crees que no es una buena política facilitar la eliminación de defectos en herramientas de gestión de defectos como *bugzilla*?

Pregunta 8. ¿Qué es la «integración continua»?

Pregunta 9. ¿Por qué es relevante la Interfaz de Usuario en las pruebas funcionales en el nivel de pruebas de sistema y aceptación?

Pregunta 10. ¿Para qué tipo de pruebas son útiles las herramientas de analítica Web? Justifica tu respuesta.

EJERCICIO (2 puntos)

Diseñad pruebas para el siguiente código utilizando la técnica de pruebas de caminos con **nivel de profundidad igual a 3**:

1. Obtened primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/expresiones y nodos que se indica en el propio código. Notad que los nodos G y J corresponden con más de una instrucción (0,25 puntos).
2. Tras haber realizado el grafo de flujo, y a la vista del código, identificad un par de aristas adyacentes al nodo H cuya ejecución consecutiva es imposible y explicad por qué (0,25 puntos).
3. Aplicad la técnica de pruebas de caminos con profundidad 3 para obtener el conjunto de caminos completos correspondiente. No consideréis como situación de prueba el par de aristas de ejecución consecutiva imposible identificado en el punto anterior (0,75 puntos).
4. Finalmente, estableced casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,75 puntos).

```
/**
 * @param n - un entero cualquiera
 * @return «true» si y solo si «n» es un número primo (es decir,
 *         si es positivo y tiene exactamente dos divisores: 1 y n.
 */
public static boolean esPrimo(int n) {
  A boolean esPrimo;
  B if (n == 2) {
    C esPrimo = true; // «n» es igual a 2, luego es primo
  }
  else if (D n < 2 || E n % 2 == 0) {
    F esPrimo = false; // «n» es menor que 2 o divisible por 2
  }
  else {
    // Se buscan posibles divisores impares de «n»
    G boolean divisorEncontrado = false;
    H int divisor = 3; // Primer divisor impar a probar
    while (H !divisorEncontrado && I divisor * divisor <= n) {
      J divisorEncontrado = n % divisor == 0;
      divisor = divisor + 2;
    }
    K esPrimo = !divisorEncontrado;
  }
  L return esPrimo;
}
```